SISTEMAS OPERATIVOS

Aula Teórica 11

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

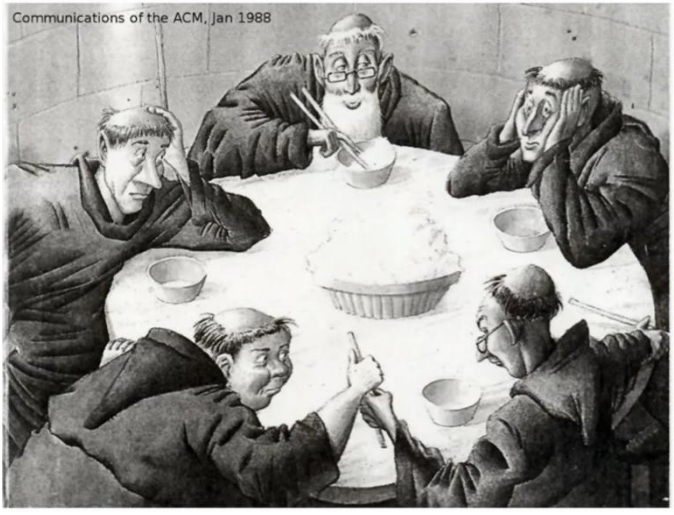
Dining Philosophers (1)

The Dining Philosophers Problem
(Dijkstra, 1965)

5 Philosophers
5 forks
Rice/spaguetti for all

Rules:
Do for life
    Philosophers think;
    When he gets sufficiently hungry
    he tries to acquire his left fork,
    and his right fork, one at a time;
    If sucessful in acquire both forks,
        he eats for a while,
        and then release both forks;
End do

Problems: DEADLOCK, and STARVATION



Dining Philosophers (2)

```
while(true){
   think();
   take_forks();
   eat();
   release_forks();
}
```

Critical zone?

2 resources:
- Left_fork;
- Right_fork.

```
#define N 5                                  /* number of philosophers */

void philosopher(int i)                      /* i: philosopher number, from 0 to 4 */
{
     while (TRUE) {
          think( );                          /* philosopher is thinking */
          take_fork(i);                      /* take left fork */
          take_fork((i+1) % N);              /* take right fork; % is modulo operator */
          eat( );                            /* yum-yum, spaghetti */
          put_fork(i);                       /* put left fork back on the table */
          put_fork((i+1) % N);               /* put right fork back on the table */
     }
}
```

Figure 6-4. A nonsolution to the dining philosophers problem.

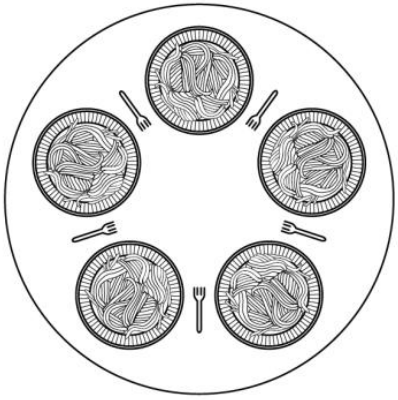


Figure 6-3. Lunch time in the Philosophy Department.

Dining Philosophers - Naive Algorithm (1)

```
while(true){
    think();
   take_forks();
   eat();
   release_forks();
}
```

First NAIVE algorithm : JAVA implementation with 2 binary semaphores (mutex), one for each resource (fork).

```
public class Filosofo extends Thread {
   private int numero;
   private Semaphore garfoEsquerdo;
   private Semaphore garfoDireito;

   public Filosofo(int numero,
                   Semaphore garfoEsq,
                   Semaphore garfoDir) {
      this.numero = numero;
      this.garfoEsquerdo = garfoEsq;
      this.garfoDireito = garfoDir;
   }
  public void run() {...}
  private void pensar() {
      System.out.println("Filosofo " + numero + " pensou.");
   }
  private void comer() {
      System.out.println("Filosofo " + numero + " comeu.");
   }
```

```
@Override
public void run() {
   try {
      while (true) {
         pensar();
         garfoEsquerdo.acquire();
         garfoDireito.acquire();
         comer();
         garfoEsquerdo.release();
         garfoDireito.release();
      }
   } catch (InterruptedException e) {
      ...
   }
}
```

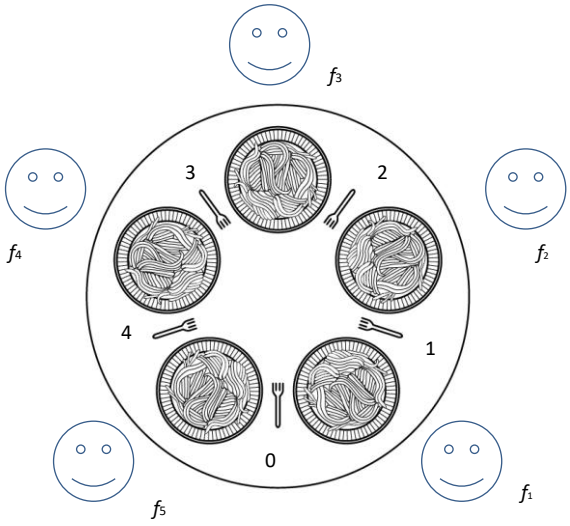
Dining Philosophers - Naive Algorithm (2)

```
public void run() {
   try {
      while (true) {
         pensar();
         garfoEsquerdo.acquire();
         garfoDireito.acquire();
         comer();
         garfoEsquerdo.release();
         garfoDireito.release();
      }
   } catch (InterruptedException e) {
      ...
   }
}
```

- Solution with 1 Philosopher (just one thread)
- Solution with 2 Philosophers at the same time (2 threads)
- Solution with 3 Philosophers at the same time (3 threads)

- Solution with 5 Philosophers at the same time (5 threads)

<< **DEADLOCK** >>



Deadlock (1)

Defenition of **Deadlock**:

*A set of processes is **deadlock** if each process in the set is waitying for na event that only another process in the set can cause.*

Because all the processes are waiting, none of them will ever cause any event that could wake up any other member of the set, and all the processes continue waiting forever.
For this model, we assume that processes are single threaded and no interrupts are possible to wake up a blocked process. The no-interrupts condition is needed to prevent na otherwise deadlocked process from being awakened by na alarm, and then causing events that release other processes in the set.

In most cases, the event that each process is waiting for is the release of some resource (such as a fork) currently possessed by another member of the set.

None of the processes can run, none of them can release any resource, and none of them can be awakened.

This kind of deadlock is called a **resource deadlock**.

Deadlock (2)

Conditions for **Resource Deadlocks**:

1. Mutual exclusion – Each resource is either currently assigned to exactly one process or is available;

2. Hold-and-wait – Processes currently holding resources that were granted earlier can request a resource;

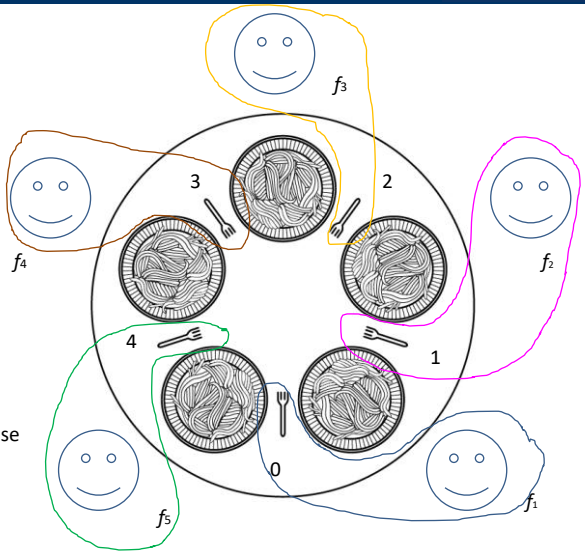
3. No-preemption – Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them;

4. Circular wait – There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member of the chain.

All of these conditions must be present for a resource deadlock to occur.

Dining Philosophers - Naive Algorithm (3)

```
public void run() {
   try {
      while (true) {
         pensar();
         garfoEsquerdo.acquire();
         garfoDireito.acquire();
         comer();
         garfoEsquerdo.release();
         garfoDireito.release();
      }
   } catch (InterruptedException e) {
      ...
   }
}
```



- Solution with 5 Philosophers at the same time (5 threads)

At the same time, all the five philosophers get access to the resouce at the left (left_fork), and all are getting blocked because none of them can get the right_fork.

<< **DEADLOCK** >>

Dining Philosophers - Naive Algorithm (4)

Second Naive algorithm

- If a philosopher can not acquire a second fork, he releases the first fork on the table.

- A timeout is applyed to the resource acquisition.

-The sleep(250) is needed to guaranty the proper release of the fork.

```
public void run() {
   try {
      while (true) {
         pensar();
         boolean comeu = false;
         do {
            garfoEsquerdo.acquire();
            boolean pegou = garfoDireito.tryAcquire(100, TimeUnit.MILLISECONDS);
            if (pegou) {
               comer(); comeu = true;
               // libera os 2 garfos
               garfoDireito.release();
               garfoEsquerdo.release();
            } else {
               // libera o primeiro garfo e aguarda
               garfoEsquerdo.release();
               sleep(250);
            }
         } while (!comeu);
      }
   } catch (InterruptedException e) {  ...  }
}
```

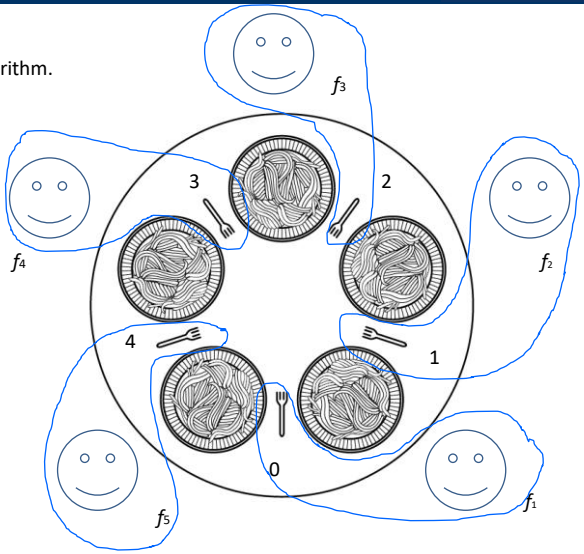
Dining Philosophers - Naive Algorithm (5)

- Solution with 5 Philosophers (5 threads) with the second algorithm.

. All philosophers gets their first fork (at the same clock pulse);

. With the tryAcquire they tried to get the second fork
(during 100 msec)

. If one of them gets the other resource he could go eating,
and release as soon as possible both forks;
Otherwise he will release the first fork,
wait for 250 msec,
and start gain;



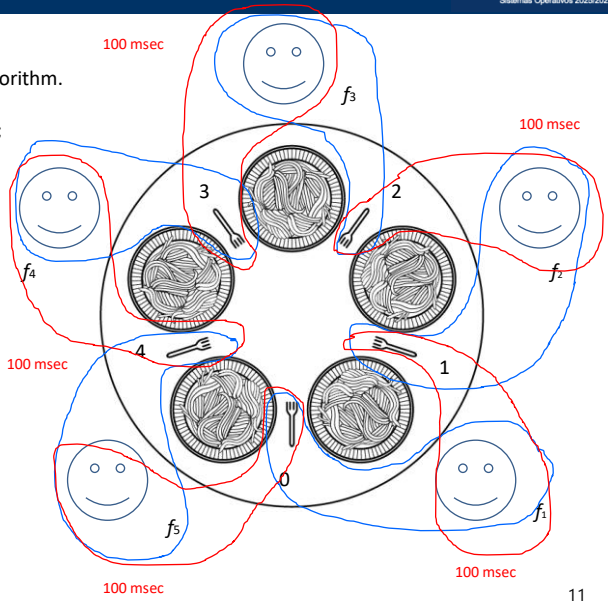
Dining Philosophers - Naive Algorithm (5)

- Solution with 5 Philosophers (5 threads) with the second algorithm.

. All philosophers gets their first fork (at the same clock pulse);

. With the tryAcquire they tried to get the second fork
(during 100 msec)

. If one of them gets the other resource he could go eating,
and release as soon as possible both forks;
Otherwise he will release the first fork,
wait for 250 msec,
and start gain;



Dining Philosophers - Naive Algorithm (5)

- Solution with 5 Philosophers (5 threads) with the second algorithm.

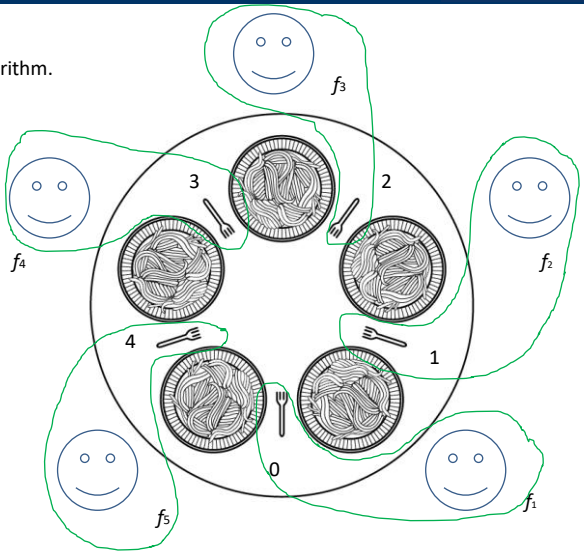
. All philosophers gets their first fork (at the same clock pulse);

. With the tryAcquire they tried to get the second fork (during 100 msec)

. If one of them gets the other resource he could go eating, and release as soon as possible both forks;
Otherwise he will release the first fork,
wait for 250 msec,
and start gain;.

These running threads never stop running.

<< **STARVATION** >>



Dining Philosophers - Naive Algorithm (6)

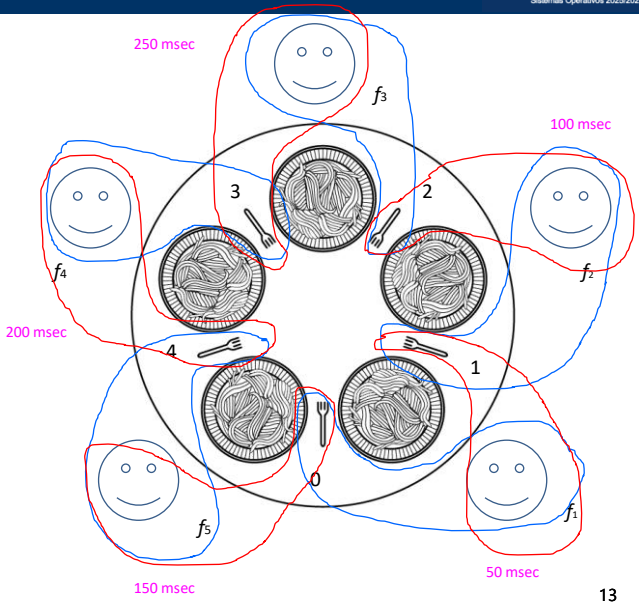
Third Naive algorithm

- If a philosopher can not acquire a second fork, he releases the first fork on the table.

- A timeout is applyed to the resource acquisition.

- Introdution of a **random sleep**.
  The philosophers would wait random time after failing to acquire the right-hand fork.

Example :
sleep(50) for philosopher i
sleep(100) for philosofer i+1
Sleep(250) for philosopher i+2
…
The philosopher with the smallest sleep will probably release first his resource, giving the turn to someone else, with the resource available.



Dining Philosophers - Naive Algorithm (7)

Another Algorithm

- Just change the order of the mutexes for one of the philosophers and it will work correctly.

But it is not an optimized solution.

```
while(true){
    pensar(p);
    if p==1{
        garfoEsquerdo.acquire();
        garfoDireito.acquire();
    }
    else{
        garfoDireito.acquire();
        garfoEsquerdo.acquire();
    }
    comer(p)
    garfoDireito.release();
    garfoEsquerdo.release();
}
```

Dining Philosophers (3)

```
#define N              5              /* number of philosophers */
#define LEFT           (i+N–1)%N      /* number of i's left neighbor */
#define RIGHT          (i+1)%N        /* number of i's right neighbor */
#define THINKING       0              /* philosopher is thinking */
#define HUNGRY         1              /* philosopher is trying to get forks */
#define EATING         2              /* philosopher is eating */
typedef int semaphore;                /* semaphores are a special kind of int */
int state[N];                         /* array to keep track of everyone's state */
semaphore mutex = 1;                  /* mutual exclusion for critical regions */
semaphore s[N];                       /* one semaphore per philosopher */

void philosopher(int i)               /* i: philosopher number, from 0 to N–1 */
{
     while (TRUE) {                   /* repeat forever */
          think( );                   /* philosopher is thinking */
          take_forks(i);              /* acquire two forks or block */
          eat( );                     /* yum-yum, spaghetti */
          put_forks(i);               /* put both forks back on table */
     }
}
```



Tanenbaum solution with N+1 semaphores.

Dining Philosophers (4)


```
void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
     down(&mutex);                      /* enter critical region */
     state[i] = HUNGRY;                 /* record fact that philosopher i is hungry */
     test(i);                           /* try to acquire 2 forks */
     up(&mutex);                        /* exit critical region */
     down(&s[i]);                       /* block if forks were not acquired */
}
void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
     down(&mutex);                      /* enter critical region */
     state[i] = THINKING;               /* philosopher has finished eating */
     test(LEFT);                        /* see if left neighbor can now eat */
     test(RIGHT);                       /* see if right neighbor can now eat */
     up(&mutex);                        /* exit critical region */
}

void test(i)                            /* i: philosopher number, from 0 to N–1 */
{
     if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
          state[i] = EATING;
          up(&s[i]);
     }
}
```

The Sleeping Barber Problem (1)



The Sleeping Barber Problem (2)


```
#define CHAIRS 5                    /* # chairs for waiting customers */

typedef int semaphore;              /* use your imagination */

semaphore customers = 0;            /* # of customers waiting for service */
semaphore barbers = 0;              /* # of barbers waiting for customers */
semaphore mutex = 1;                /* for mutual exclusion */
int waiting = 0;                    /* customers are waiting (not being cut) */

void barber(void)
{
    while (TRUE) {
        down(&customers);           /* go to sleep if # of customers is 0 */
        down(&mutex);               /* acquire access to 'waiting' */
        waiting = waiting – 1;      /* decrement count of waiting customers */
        up(&barbers);               /* one barber is now ready to cut hair */
        up(&mutex);                 /* release 'waiting' */
        cut_hair( );                /* cut hair (outside critical region) */
    }
}


void customer(void)
{
    down(&mutex);                   /* enter critical region */
    if (waiting < CHAIRS) {         /* if there are no free chairs, leave */
        waiting = waiting + 1;      /* increment count of waiting customers */
        up(&customers);             /* wake up barber if necessary */
        up(&mutex);                 /* release access to 'waiting' */
        down(&barbers);             /* go to sleep if # of free barbers is 0 */
        get_haircut( );             /* be seated and be serviced */
    } else {
        up(&mutex);                 /* shop is full; do not wait */
    }
}
```

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

